

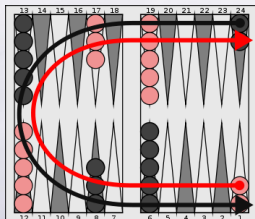
Décision séquentielle pour un agent autonome

CM1 : MDP et Planification sous incertitudes

Laëtitia Matignon



Décision séquentielle pour un agent autonome : Exemples

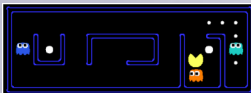


Compétences

- ≡ savoir modéliser un problème de décision séquentielle sous forme de processus décisionnel markovien (MDP)
- ≡ connaître différentes méthodes de résolution :
 - planification sous incertitudes (modèle connu) : algorithme d'itérations sur les valeurs
 - apprentissage par renforcement (modèle partiellement connu) : algorithmes Q-learning tabulaire et approximé
 - (Partie 3 : apprentissage profond par renforcement)

Séances :

- ≡ CM1 + TD1 + TP1 : Modélisation MDP et Planification sous incertitudes
- ≡ CM2 + TD2 + TP2 + TP3 : Apprentissage par renforcement



Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

Agent



VS

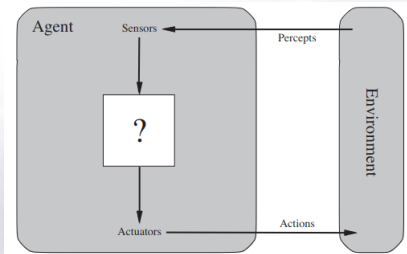


VS



Comment définir un agent ?

Agent : boucle perception/action



- ≡ Interaction avec l'environnement
- ≡ O : unité élémentaire de perception (via des capteurs)
- ≡ A : un ensemble d'actions (via des actionneurs)

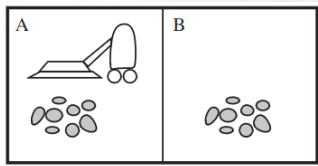
- ≡ L'agent possède une fonction de décision $f_d : (O_n) \rightarrow A$ *An agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived¹*

Définition

« Un agent est une entité **autonome**, réelle ou abstraite, qui est capable d'**agir** sur elle-même et sur son environnement, qui, dans un univers multiagent, peut **communiquer** avec d'autres agents, et dont le comportement est la conséquence de ses **observations**, de ses connaissances et des **interactions** avec les autres agents. » [Ferber, Les systèmes multi-agents, vers une intelligence collective, 1995]

1. Artificial Intelligence - A Modern Approach, Stuart Russell and Peter Norvig, 1995

Exemple Simple : agent aspirateur



Vacuum agent

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

- ▢ 2 localisations : A et B
- ▢ Perceptions : localisation $\in \{A, B\}$ et contenu $\in \{Dirty, Clean\}$, exemple d'une perception : [A ;Dirty]
- ▢ Actions : Droite, Gauche, Aspirer, Ne rien faire
- ▢ Une fonction de décision possible : Si la localisation courante est sale, alors aspirer, sinon aller à l'autre localisation

Agent rationnel

Comment définir un bon comportement ?

Définition

For each possible percept sequence, a **rational agent** should select an action that is expected to **maximize its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has^a.

a. Artificial Intelligence - A Modern Approach, Stuart Russell and Peter Norvig, 1995

- ≡ Mesure de performance : évalue le comportement de l'agent dans l'environnement
- ≡ Définir ce que l'on veut obtenir plutôt que comment on pense que l'agent doit se comporter

Mesure de performance de l'agent aspirateur

un point par case nettoyée à chaque pas de temps plutôt que quantité de poussière ramassée

Agent rationnel et environnement

Pour modéliser un agent rationnel, on doit spécifier l'environnement où doit se dérouler la tâche de l'agent.

- **L'environnement** représente le **problème** à résoudre
- **L'agent** représente la **solution**
- On doit définir l'environnement de la tâche (PEAS)

PEAS (Performance, Environment, Actuators, Sensors)

- Définir la mesure de performance (notion de compromis en cas de conflits dans les objectifs à atteindre)
- Définir l'environnement
- Définir les capteurs
- Définir les effecteurs

Exemple Simple : Vacuum agent

PEAS pour l'agent aspirateur

- ▮ Mesure de performance : un point par case nettoyée à chaque pas de temps
- ▮ Environnement :
 - connu (2 localisations A et B, A à gauche de B)
 - mais emplacement initial de l'agent et distribution de la poussière inconnue ;
 - une localisation nettoyée reste propre ;
- ▮ Capteurs : L'agent perçoit correctement la case où il se trouve et sa propreté
- ▮ Actions : Droite, Gauche, Aspirer, Ne rien faire

Dans ces circonstances, l'agent Reflex avec la fonction de décision ci-dessous est rationnel car il va maximiser sa mesure de performance.

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Exemple Simple : Vacuum agent

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action  
  if status = Dirty then return Suck  
  else if location = A then return Right  
  else if location = B then return Left
```

Mais sera-t-il rationnel dans d'autres PEAS ?

- ≡ Si la mesure de performance inclue une pénalité pour chaque déplacement ?
- ≡ Si l'environnement peut changer (de la poussière peut ré-apparaître dans une localisation nettoyée) ?
- ≡ Si l'environnement n'est pas connu au départ ?

Propriétés de l'environnement

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous

Observabilité :

- Entièrement observable : l'agent a accès à toute l'information pertinente sur l'environnement (on parle aussi d'état)
- Partiellement observable : par ex. perceptions de l'agent aspirateur $\in \{Dirty, Clean\}$ mais **ne perçoit pas** localisation $\in \{A, B\}$

Propriétés de l'environnement

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous

Stochasticité :

- Déterministe : L'état prochain de l'environnement est complètement déterminé par l'état courant et l'action exécutée par l'agent
- Stochastique ex. : si l'agent aspirateur fait Droite dans la localisation A, alors l'état prochain sera B avec 80% et A avec 20% ; si l'action d'aspiration ne marche pas tout le temps ; ...

Propriétés de l'environnement

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous

Séquentialité :

- Episodique : Le choix de l'action dépend seulement de la perception courante (one-shot actions) ; par exemple les tâches de classification
- Séquentiel : La décision courante peut affecter les décisions futures ; l'agent a besoin de mémoire pour déterminer les prochaines meilleures actions
- On peut avoir une prise de décision séquentielle à l'intérieur de différents épisodes indépendants ...

Propriétés de l'environnement

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous

- Statique (vs. Dynamique) : un environnement dynamique change pendant la délibération de l'agent
- Discret (vs. Continu) : pour les états, le temps, les perceptions, les actions.
- Mono-agent vs multi-agent

Le monde réel est partiellement observable, non déterministe, séquentiel, dynamique, continu, multi-agent.

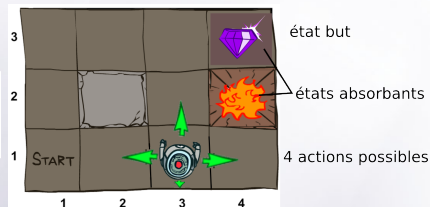
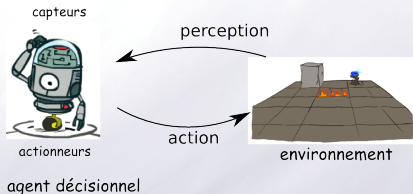
Conclusion

- Les agents interagissent avec leur environnement à travers une boucle de perception-action
- L'environnement de la tâche doit être spécifié (PEAS)
- La mesure de performance évalue le comportement de l'agent dans son environnement
- Un agent parfaitement rationnel maximise la performance attendue

Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

Problème posé



Exemple fil rouge

PEAS exemple fil rouge

- ▮ Mesure de performance : atteindre le plus rapidement possible l'état objectif depuis l'état initial
- ▮ Environnement connu
- ▮ Capteurs : L'agent perçoit correctement la case où il se trouve
- ▮ Actions : 4 directions cardinales (si mur, reste sur place)

Idée d'algorithmes ?

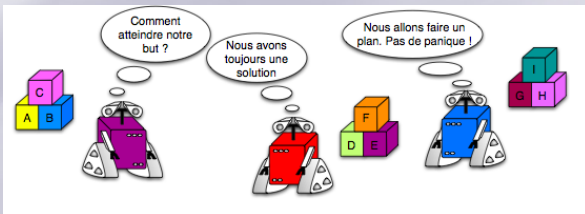
Problème de planification

Définition

*Planning is the reasoning side of acting. It is an abstract, explicit deliberation process that **chooses and organizes actions** by **anticipating their expected outcomes**. This deliberation aims at achieving as best as possible some **pre-stated objectives**.* [Automated Planning, M. Ghallab et al. Morgan Kaufmann, 2004]

Planifier

Trouver un **plan** ou **une séquence d'actions** pour aller d'un état initial à un état but en respectant certains objectifs.

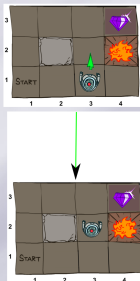


Le monde réel n'est pas parfait !

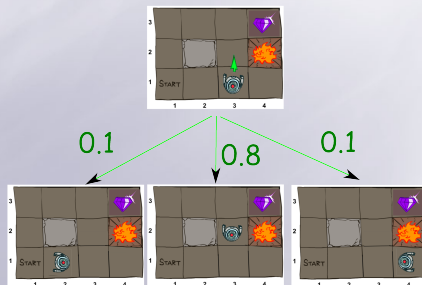
Environnement stochastique (non-déterministe)

Le résultat d'une action est incertain.

environnement
déterministe

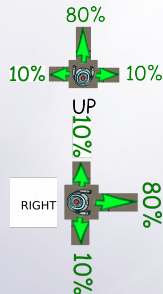
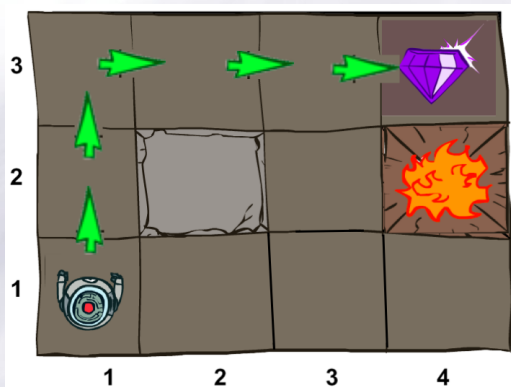


environnement stochastique



Le monde réel n'est pas parfait !

On modifie les règles : toutes les actions sont stochastiques

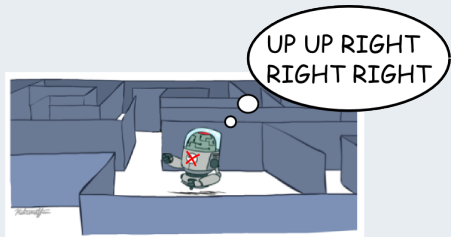
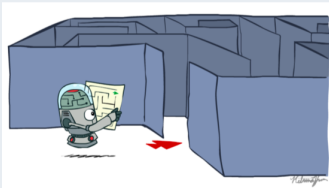


idem pour
LEFT et DOWN
(4 actions non-déterministes)

Quelle est la probabilité que le plan trouvé par A* (HAUT HAUT DROITE DROITE DROITE) atteigne le but dans cet environnement stochastique ?

Limites de la planification a priori

2 phases en planification

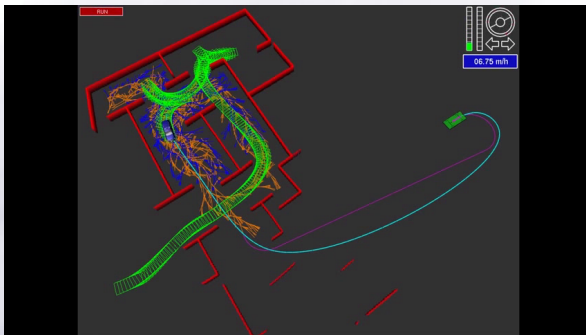


1- Planification : calcul d'un plan 2- Exécution du plan

- ≡ A* exécute en aveugle !
- ≡ A* suppose que l'environnement est déterministe.

Utiliser la boucle perception-action à l'exécution !

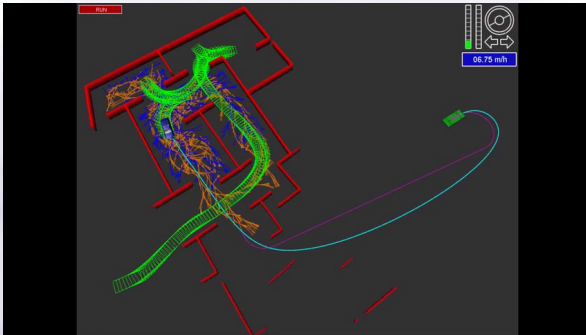
Planification sous incertitudes



Exemple de *re-planning* (DARPA challenge)

- ≡ entrelacer planification et exécution : *re-planning*
- ≡ utiliser un modèle qui prend en compte les incertitudes et planifie une seule fois

Planification sous incertitudes



Exemple de *re-planning* (DARPA challenge)

- ≡ entrelacer planification et exécution : *re-planning*
- ≡ utiliser un modèle qui prend en compte les **incertitudes** et planifie une seule fois

Quelques domaines d'applications

- Aéronautique
- Militaire
- Robotique industrielle, transport
- Vie de tous les jours (aspirateur automatique, tondeuse automatique)
- Informatique (Jeux vidéo, Informatique ambiante)



Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

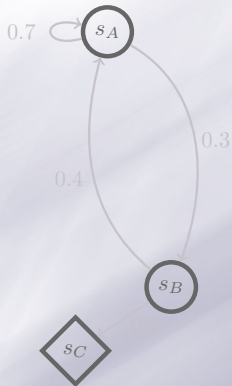
Formalisation mathématique

- Environnement stochastique
- On va tout d'abord définir le **problème** : modèle MDP (*Markov Decisional Process*)
- On va ensuite définir sa **solution** : une politique
- On va ensuite définir l'**objectif** : trouver une politique optimale

Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

Environnement stochastique sans agent



Chaîne de Markov

- ensemble fini d'états : S
- fonction de transition $T : S \times S \rightarrow [0; 1]$
 $T(s, s') = P(s_{t+1} = s' | s_t = s)$

Propriété de Markov (sans mémoire)

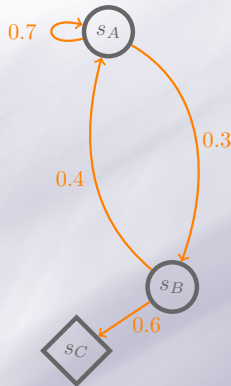
Les transitions ne dépendent que de l'état actuel :
 $P(s_{t+1} | s_t, s_{t-1}, \dots, s_0) = P(s_{t+1} | s_t)$

Exemple d'un épisode :

$s_A, s_B, s_A, s_A, s_B, s_C$

Ajoutons une composante décisionnelle pour modéliser un agent.

Environnement stochastique sans agent



Chaîne de Markov

- ≡ ensemble fini d'états : S
- ≡ fonction de transition $T : S \times S \rightarrow [0; 1]$
 $T(s, s') = P(s_{t+1} = s' | s_t = s)$

Propriété de Markov (sans mémoire)

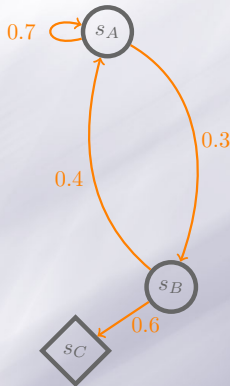
Les transitions ne dépendent que de l'état actuel :
 $P(s_{t+1} | s_t, s_{t-1}, \dots, s_0) = P(s_{t+1} | s_t)$

Exemple d'un épisode :

$s_A, s_B, s_A, s_A, s_B, s_C$

Ajoutons une composante décisionnelle pour modéliser un agent.

Environnement stochastique sans agent



Chaîne de Markov

- ≡ ensemble fini d'états : S
- ≡ fonction de transition $T : S \times S \rightarrow [0; 1]$
 $T(s, s') = P(s_{t+1} = s' | s_t = s)$

Propriété de Markov (sans mémoire)

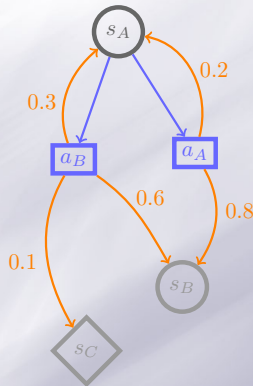
Les transitions ne dépendent que de l'état actuel :
 $P(s_{t+1} | s_t, s_{t-1}, \dots, s_0) = P(s_{t+1} | s_t)$

Exemple d'un épisode :

$s_A, s_B, s_A, s_A, s_B, s_C$

Ajoutons une composante décisionnelle pour modéliser un agent.

Environnement stochastique avec agent



Exemple d'un
épisode : $s_A, a_A, s_A,$
 a_B, s_C

Processus Décisionnel Markovien (MDP)

- ▤ ensemble fini d'états : S
- ▤ ensemble fini d'actions : A ou $A(s)$
- ▤ fonction de transition $T : S \times A \times S \rightarrow [0; 1]$
 $T(s, a, s') = P(s_{t+1} = s' | s_t = s, a_t = a)$
- ▤ ...

Propriété de Markov

Les conséquences d'une action a_t dans un état s_t (i.e. l'état d'arrivée s_{t+1}) ne dépendent que de l'état courant s_t :

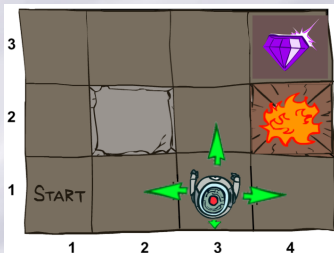
$$P(s_{t+1} | s_t, a_t, r_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1} | s_t, a_t)$$

L'agent n'agit qu'en fonction de son état courant.

Dans notre problème ...

Processus Décisionnel markovien (MDP)

- ensemble fini d'états : S
- ensemble fini d'actions : A ou $A(s)$
- fonction de transition $T : S \times A \times S \rightarrow [0; 1]$
 $T(s, a, s') = P(s_{t+1} = s' | s_t = s, a_t = a)$
 Probabilité d'arriver dans s' lorsque l'on fait a dans s



■ $|S| = ?$

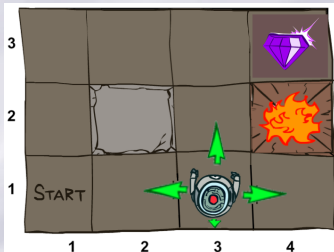
■ $A = ?$

■

Dans notre problème ...

Processus Décisionnel markovien (MDP)

- ensemble fini d'états : S
- ensemble fini d'actions : A ou $A(s)$
- fonction de transition $T : S \times A \times S \rightarrow [0; 1]$
 $T(s, a, s') = P(s_{t+1} = s' | s_t = s, a_t = a)$
 Probabilité d'arriver dans s' lorsque l'on fait a dans s

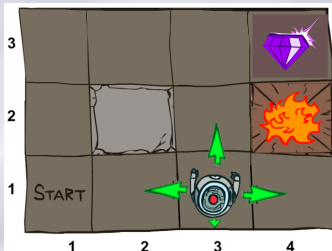


- $|S| = 11$ (cases accessibles)
- $A = UP, DOWN, RIGHT, LEFT$
-

Dans notre problème ...

Processus Décisionnel markovien (MDP)

- ▢ ensemble fini d'états : S
- ▢ ensemble fini d'actions : A ou $A(s)$
- ▢ fonction de transition $T : S \times A \times S \rightarrow [0; 1]$
 $T(s, a, s') = P(s_{t+1} = s' | s_t = s, a_t = a)$
 Probabilité d'arriver dans s' lorsque l'on fait a dans s



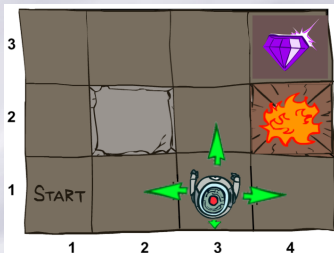
- ▢ $|S| = 11$ (cases accessibles)
- ▢ $A = UP, DOWN, RIGHT, LEFT$
- ▢ $T(1 \times 1, UP, 1 \times 2) = ?$
 $T(1 \times 1, UP, 2 \times 1) = ?$

Dans notre problème ...

Processus Décisionnel markovien (MDP)

- ≡ ensemble fini d'états : S
- ≡ ensemble fini d'actions : A ou $A(s)$
- ≡ fonction de transition $T : S \times A \times S \rightarrow [0; 1]$
 $T(s, a, s') = P(s_{t+1} = s' | s_t = s, a_t = a)$
 Probabilité d'arriver dans s' lorsque l'on fait a dans s

- ≡ $T(1 \times 1, UP, 1 \times 2) = 0.8$
 $T(1 \times 1, UP, 2 \times 1) = 0.1$
 $T(1 \times 1, UP, 1 \times 1) = 0.1$
- ≡ $T(1 \times 1, RIGHT, 2 \times 1) = 0.8$
 $T(1 \times 1, RIGHT, 1 \times 1) = 0.1 \dots$



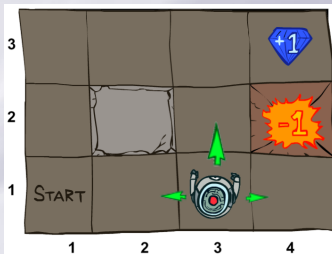
Dans notre problème ...

Définition de l'objectif

Les récompenses indiquent à quoi aboutir mais pas comment y parvenir.

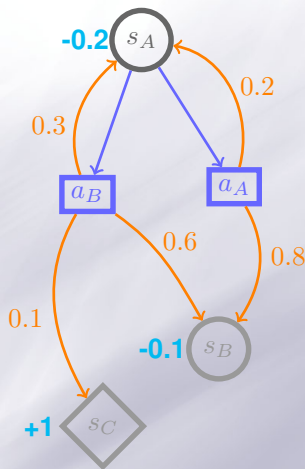
Processus Décisionnel Markovien (MDP)

- ▢ ensemble fini d'états : S
- ▢ ensemble fini d'actions : A ou $A(s)$
- ▢ fonction de transition $T : S \times A \times S \rightarrow [0; 1]$
- ▢ fonction de récompense $R : S \times A \times S \rightarrow \mathbb{R}$



- ▢ $|S| = 11$
- ▢ $A = UP, DOWN, RIGHT, LEFT$
- ▢ $R(3 \times 3, RIGHT, 4 \times 3) = 1$
- ▢ $R(3 \times 3, DOWN, 4 \times 3) = 1$
- ▢ $R(3 \times 2, RIGHT, 4 \times 2) = -1$

Conclusion : modèle MDP

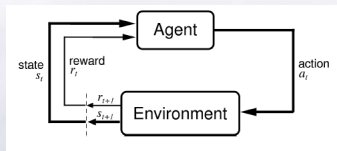


Exemple1 d'un MDP

■ Définition du problème (MDP) :

- ensemble fini d'états : S (s_A, s_B, s_C dans exemple 1)
- ensemble fini d'actions : A (a_A, a_B dans exemple 1)
- fonction de transition
 $T : S \times A \times S \rightarrow [0; 1]$ (en orange dans exemple 1)
- fonction de renforcement
 $R : S \times A \times S \rightarrow \mathbb{R}$ (en bleu dans exemple 1)

MDP et boucle perception-action

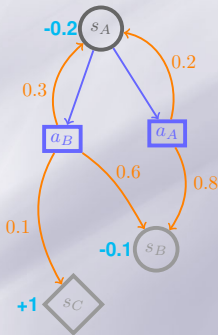
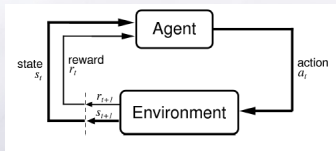


- ≡ A chaque instant t , l'agent observe son état (s_t), agit (a_t), reçoit une récompense (r_{t+1}) et transitionne dans un nouvel état (s_{t+1}).
- ≡ Un **épisode** : $s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, \dots$
 - peut se terminer (si états absorbants ou nombre d'actions max)
 - ou durer infiniment !

Objectif de l'agent

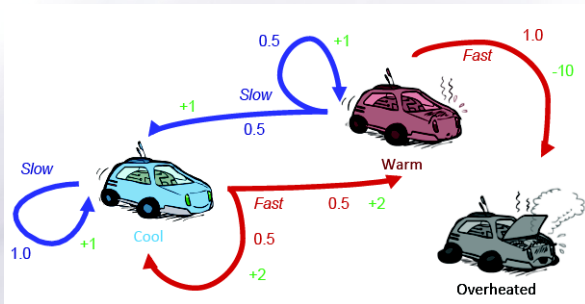
L'agent doit déterminer quelle action faire dans chaque état pour maximiser ses récompenses

MDP et boucle perception-action



Un épisode dans ce MDP :
 $s_A, a_A, r = -0.2, s_A, a_B, r = 1, s_C$

Modélisation MDP : Exemples

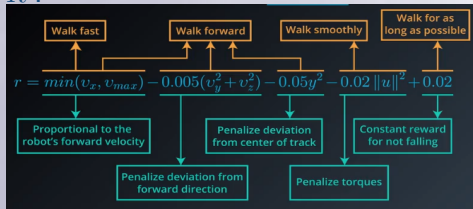


MDP à 3 états, 2 actions, R et T spécifiés dans le graph.
 Objectif : rouler vite sans surchauffer

Modélisation MDP : Exemples



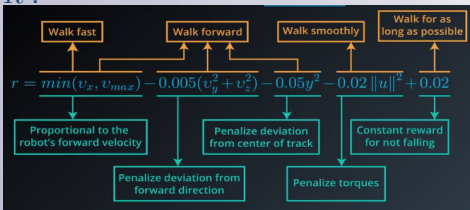
- ≡ Objectif : marcher le plus vite possible sans tomber
- ≡ A : forces appliquées aux différentes articulations
- ≡ S :
 - positions et vitesses des articulations
 - caractéristiques du terrain
 - contact au sol
- ≡ T : simulateur physique
- ≡ R :



Modélisation MDP : Exemples



- ≡ Objectif : marcher le plus vite possible sans tomber
- ≡ A : forces appliquées aux différentes articulations
- ≡ S :
 - positions et vitesses des articulations
 - caractéristiques du terrain
 - contact au sol
- ≡ T : simulateur physique
- ≡ R :



Exercice du TD : modélisation sous forme d'un MDP

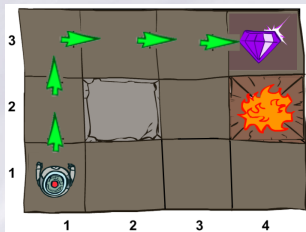
Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 **Formalisation mathématique**
 - Problème : Modèle MDP
 - **Solution : Politique**
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

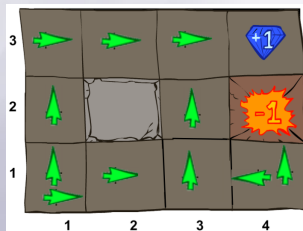
Solution d'un MDP

Politique π

L'agent doit déterminer quelle **action faire dans chaque état** : politique $\pi : S \rightarrow A$ associe une (ou plusieurs) action(s) à exécuter dans chaque état



Plan pour un environnement déterministe



Politique pour un environnement stochastique :

$$\pi(1 \times 1) = \{UP, RIGHT\}$$

$$\pi(1 \times 2) = UP$$

...

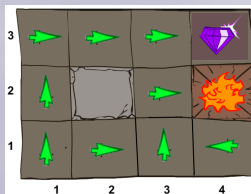
Familles de politiques pour les MDP

politique π	déterministe	stochastique
markovienne	$s \rightarrow a$	$s, a \rightarrow [0, 1]$
histoire-dépendante	$h \rightarrow a$	$h, a \rightarrow [0, 1]$

$h_t = (s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t)$ historique à t

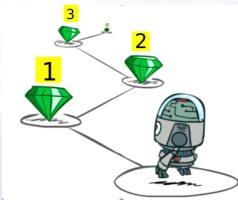
Politiques stationnaires

- Si le choix de la meilleure décision à prendre dépend de l'instant t , la politique est non-stationnaire (π dépend de t).
- Sinon la politique est **stationnaire** $\forall t \pi_t = \pi$



politique markovienne déterministe stationnaire $\pi : S \rightarrow A$

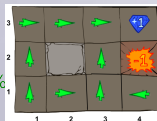
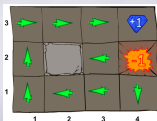
Horizon et politique non stationnaire



Horizon

Nombre de pas de temps sur lesquels l'agent raisonne pour prendre ses décisions. Peut être fini ou infini

- Avec horizon infini, la politique est stationnaire $\pi(s) \rightarrow a$
- Avec horizon fini, la politique n'est plus stationnaire $\pi(s, t) \rightarrow a$



$R(s) = 0.0 \forall s$ non absorbant

Horizon fini : π^* évolue au cours du temps donc non-stationnaire !

On va s'intéresser au cas avec horizon infini et politique stationnaire.

Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

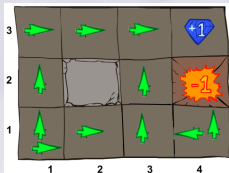
Solution optimale d'un MDP

Objectif de l'agent

L'agent doit déterminer quelle action faire dans chaque état pour **maximiser ses récompenses** :

≡ à t , si on suit π , on a la récompense cumulée

$$G_t = r_{t+1} + r_{t+2} + \dots = \sum_{t=0}^{\infty} r_{t+1}$$



Politique π

A t , l'agent est dans 1×3 , et suit π , les récompenses cumulées possibles sont (car les actions sont stochastiques) :

$$\equiv 0 + 0 + 1$$

$$\equiv 0 + 0 + 0 + 1$$

$$\equiv 0 + 0 + 0 + (-1)$$

$$\equiv \dots$$

Espérance de récompenses $E_{\pi}[G_t]$

On ne connaît pas avec certitudes G_t (futur+stochasticité) : on parle d'**espérance** de récompenses cumulées, car ce sont les récompenses cumulées que l'on espère obtenir **à travers la séquence d'états futurs**.

Solution optimale d'un MDP

Politique optimale π^*

Résoudre un MDP consiste à trouver une politique **optimale** notée π^* qui donne pour tout état, **l'action** permettant de **maximiser l'espérance de récompenses cumulées**.

Une politique optimale maximise un critère de performance donné, ici, c'est le critère total :

$$G = \sum_{t=0}^{\infty} r_{t+1}$$

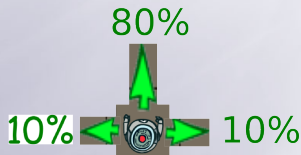
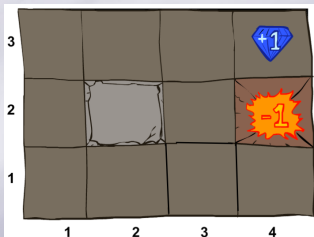
Attention : π^* dépend de R !

Il faut bien choisir les récompenses ...

Dans notre problème ...

La politique optimale $\pi^* : S \rightarrow A$ donne, pour chaque état, l'action permettant de **maximiser l'espérance de récompenses cumulées** : $G = \sum_{t=0}^{\infty} r_{t+1}$

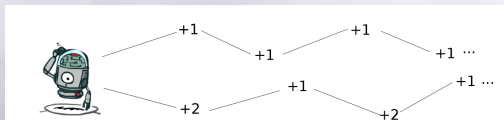
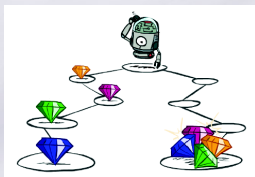
Quelle serait la politique optimale dans notre labyrinthe (avec $R(*, *, s) = 0 \ \forall s$ non absorbant) ?



Pondération du cumul de récompenses

Horizon infini

Solution 2 : changer le critère à optimiser



Temporal credit assignment problem

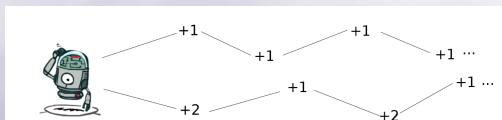
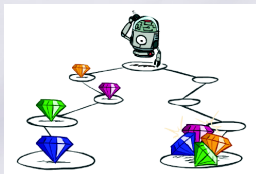
Quelle séquence de récompenses est préférée selon le critère total

$$G = \sum_{t=0}^{\infty} r_{t+1} ?$$

Pondération du cumul de récompenses

Horizon infini

Solution 2 : changer le critère à optimiser



A horizon infini (ou très grand), un compromis est nécessaire entre récompenses immédiates et futures pour borner G et choisir entre différentes trajectoires.

Récompenses pondérées

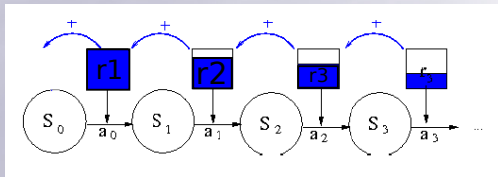
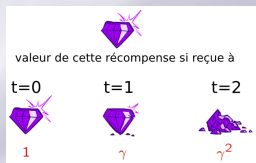
Compromis nécessaire entre récompenses immédiates et futures :
pondération des récompenses selon leur éloignement dans le futur.

$$G_t^\gamma = \gamma^0 * r_{t+1} + \gamma^1 * r_{t+2} + \gamma^2 * r_{t+3} + \dots$$

avec $\gamma \in [0, 1[$

Par ex. avec $\gamma = 0.9$:

$$G_t = 1 * r_{t+1} + 0.9 * r_{t+2} + 0.81 * r_{t+3} + \dots$$

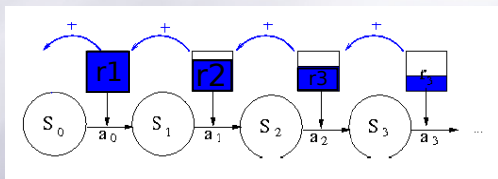
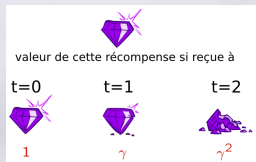


Récompenses pondérées

Critère pondéré G^γ

Cumul des récompenses avec facteur d'atténuation $\gamma \in [0; 1[$

$$\equiv G^\gamma = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$$



γ règle l'importance des récompenses futures vs. récompenses immédiates.

$$\equiv \gamma = 0 : G = r_1$$

agent cigale qui maximise la récompense immédiate (horizon de 1)

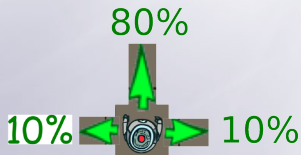
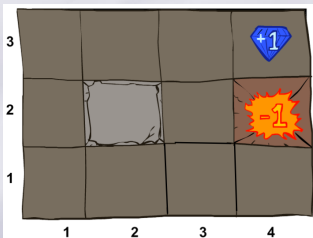
$$\equiv \gamma \rightarrow 1 : G = r_1 + r_2 + r_3 + \dots$$

agent fourmi qui sacrifie petit gain à court terme pour privilégier meilleur gain à long terme (horizon grand)

$$\equiv \gamma < 1 \implies G \leq \frac{R_{max}}{1-\gamma}$$

Dans notre problème ...

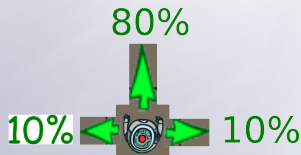
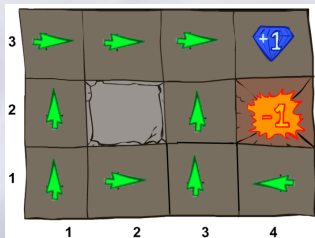
Quelle serait la politique optimale avec un critère pondéré
 $G^\gamma = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$, $\gamma = 0.9$ et $R(*, *, s) = 0 \forall s$ non absorbant ?



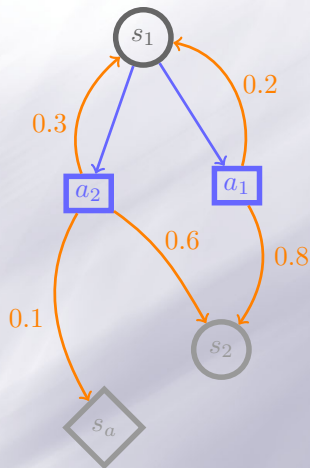
Dans notre problème ...

Quelle serait la politique optimale avec un critère pondéré

$G^\gamma = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$, $\gamma = 0.9$ et $R(*, *, s) = 0 \ \forall s$ non absorbant ?



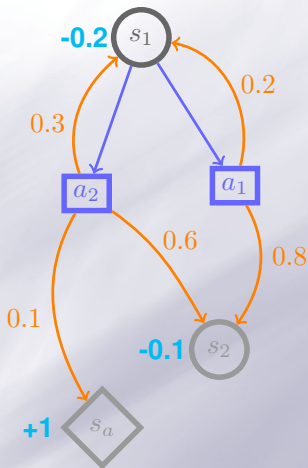
Résumons ...



Processus Décisionnel Markovien

- Environnement stochastique (avec actions incertaines)
- Récompense r_t à chaque pas de temps (formalise l'objectif)
- Solution = politique optimale π^*
- Maximise critère pondéré avec γ : récompenses futures vs immédiates

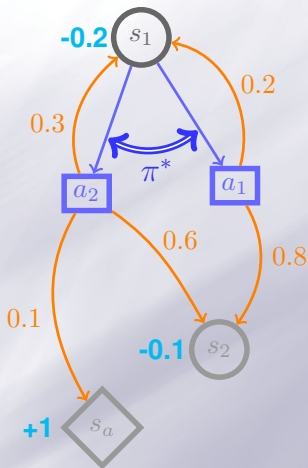
Résumons ...



Processus Décisionnel Markovien

- Environnement stochastique (avec actions incertaines)
- Récompense r_t à chaque pas de temps (formalise l'objectif)
- Solution = politique optimale π^*
- Maximise critère pondéré avec γ : récompenses futures vs immédiates

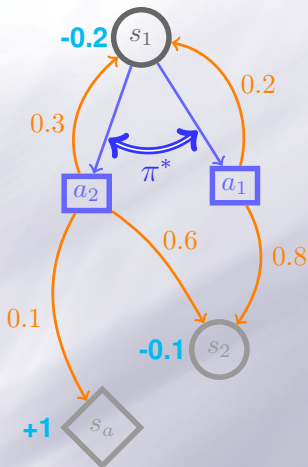
Résumons ...



Processus Décisionnel Markovien

- Environnement stochastique (avec actions incertaines)
- Récompense r_t à chaque pas de temps (formalise l'objectif)
- Solution = politique optimale π^*
- Maximise critère pondéré avec γ : récompenses futures vs immédiates

Résumons ...

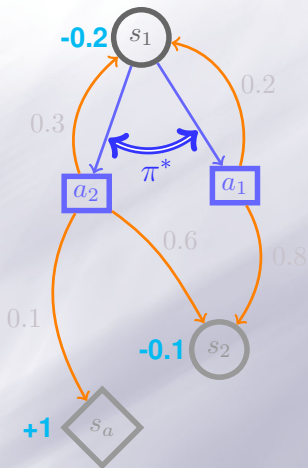


Processus Décisionnel Markovien

- Environnement stochastique (avec actions incertaines)
- Récompense r_t à chaque pas de temps (formalise l'objectif)
- Solution = politique optimale π^*
- Maximise critère pondéré avec γ : récompenses futures vs immédiates

Les politiques obtenues sont plus robustes aux incertitudes que les plans obtenus par des méthodes déterministes

Résumons ...



Processus Décisionnel Markovien

- Environnement stochastique (avec actions incertaines)
- Récompense r_t à chaque pas de temps (formalise l'objectif)
- Solution = politique optimale π^*
- Maximise critère pondéré avec γ : récompenses futures vs immédiates

Les politiques obtenues sont plus robustes aux incertitudes que les plans obtenus par des méthodes déterministes

Comment calculer la politique optimale ?

Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

Fonction de valeur

La politique optimale $\pi^* : S \rightarrow A$ donne pour chaque état l'action permettant de **maximiser l'espérance de récompenses cumulées** :

$$G = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$$

Fonction de valeur V^π

La valeur $V^\pi(s)$ d'un état s selon une politique π est l'**espérance** de récompenses cumulées G si l'agent **démarre dans l'état s** et suit la politique π :

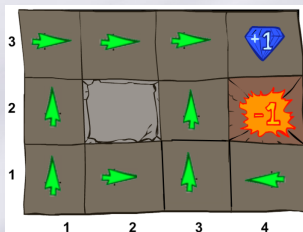
$$V^\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right\}$$

Illustration de la fonction de valeur

Fonction de valeur V^π

La valeur $V^\pi(s)$ d'un état s selon une politique π est l'**espérance** de récompenses cumulées G si l'agent **démarre dans l'état s** et suit la politique π :

$$V^\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right\}$$

 π

0,8100	0,9000	1,0000	
0,7290		0,9000	
0,6561	0,7290	0,8100	0,7290

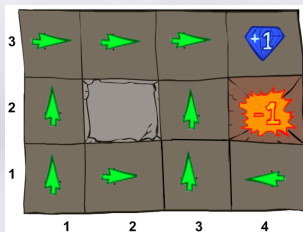
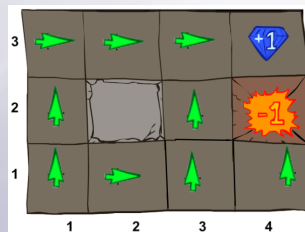
 V^π

(exemple avec un environnement déterministe)

- Dans $s = 3 \times 3$, si suit π , $V^\pi(3 \times 3) = 1$
- Dans $s = 2 \times 3$, si suit π , $V^\pi(2 \times 3) = 0 + \gamma * 1 = 0.9$
- Dans $s = 1 \times 3$, si suit π , $V^\pi(1 \times 3) = 0 + \gamma * 0 + \gamma^2 * 1 = 0.81$

Intérêts de la fonction de valeur

Comparaison de politiques : $\pi' \geq \pi$ ssi $V_{\pi'}(s) \geq V_{\pi}(s)$ pour tout $s \in S$


 π'
 $\geq$

 π

$$V^{\pi'}(4 \times 1) \geq V^{\pi}(4 \times 1)$$

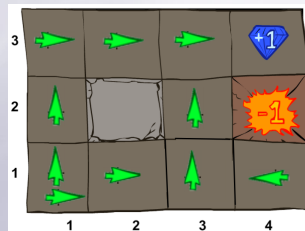
Intérêts de la fonction de valeur

- ▮ Comparaison de politiques : $\pi' \geq \pi$ ssi $V_{\pi'}(s) \geq V_{\pi}(s)$ pour tout $s \in S$
- ▮ Définition de la politique optimale : la politique optimale π^* satisfait $\pi^* \geq \pi$ pour tout π
- ▮ La fonction de valeur optimale $V^* = V^{\pi^*} = \max_{\pi} V^{\pi}$

Intérêts de la fonction de valeur

- Si on connaît V^π on peut calculer π :
 - Dans chaque état on choisit l'action qui va maximiser le retour espéré

0,8100	0,9000	1,0000	
0,7290		0,9000	
0,6561	0,7290	0,8100	0,7290



A partir de V^π on peut calculer π
(exemple avec un environnement déterministe)

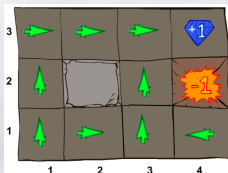
Intérêts de la fonction de valeur

- ▮ Pour trouver la politique optimale, on va chercher la fonction de valeur optimale V^*
- ▮ On pourra alors calculer π^* à partir de V^*

Comment calculer V^* ? On va utiliser l'équation de Bellman

Propriété fondamentale de V

$$V^\pi(s) = E_\pi\{\sum_{t=0}^{\infty} \gamma^t r_{t+1} | s_0 = s\}$$

 π

0,8100	0,9000	1,0000	
0,7290		0,9000	
0,6561	0,7290	0,8100	0,7290

 V^π

Valeur de $s = 2 \times 3$ si suit π :

$$V^\pi(2 \times 3) = 0 + \gamma * 1 = 0.9 \quad \text{OU} \quad V^\pi(2 \times 3) = 0 + \gamma * V^\pi(3 \times 3) = 0 + \gamma * 1 = 0.9$$

Valeur de $s = 1 \times 3$ si suit π :

$$V^\pi(1 \times 3) = 0 + \gamma * 0 + \gamma^2 * 1 = 0.81 \quad \text{OU} \quad V^\pi(1 \times 3) = 0 + \gamma * V^\pi(2 \times 3) = 0 + \gamma * 0.9 = 0$$

$$V^\pi(s) = \text{la récompense immédiate} + \gamma * \text{valeur de l'état suivant}$$

Propriété fondamentale de V

Equation de Bellman

Elle définit la valeur d'un état en fonction de la valeur des états lui succédant :

$$\begin{aligned}
 V^\pi(s) &= E\left\{\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid \pi, s_0 = s\right\} = E\left\{r_1 + \gamma \sum_{t=0}^{\infty} \gamma^t r_{t+2} \mid \pi, s_0 = s\right\} \\
 &= \sum_{s' \in S} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma E\left\{\sum_{t=0}^{\infty} \gamma^t r_{t+2} \mid \pi, s_1 = s'\right\}] \\
 V^\pi(s) &= \sum_{s' \in S} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]
 \end{aligned}$$

$V^\pi(s)$ = la récompense immédiate + γ * valeur de l'état suivant
avec prise en compte de la stochasticité

Intérêts de la fonction de valeur

- ≡ Pour trouver la politique optimale, on cherche la fonction de valeur optimale V^* pour en extraire π^*

Comment calculer V^* ? On va utiliser l'équation de Bellman

Equation d'optimalité de Bellman

V^* est l'unique solution de

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Comment calculer V^*

- ≡ Résoudre le système de $n = |S|$ équations non-linéaires, n inconnues
- ≡ Approximation par itération

Principe d'optimalité de Bellman

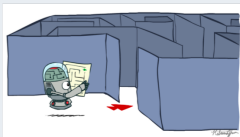
- ≡ Si l'on sait trouver la solution optimale (politique) à partir de l'étape $t + 1$ quel que soit l'état s_{t+1} , alors on peut trouver la décision optimale à l'étape t pour tout état possible s_t (et donc travailler par récurrence).

Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP**
- 6 Application à l'exploration

Rappels : planification

2 phases en planification



Phase 1 : planification



Phase 2 : exécution

- ≡ phase 1 : **hors-ligne**, planification : calcul de la politique π , l'agent **n'agit pas** dans l'environnement
- ≡ phase 2 : **en-ligne**, exécution de la politique : l'agent **agit** dans l'environnement en suivant la politique calculée
 - Boucle perception/action : perçoit (s_t, r_t) , décide $(\pi(s_t))$ et agit (a_t)
 - processus séquentiel $s_0, a_0, s_1, r_1, a_1, s_2, r_2, a_2, \dots$

Itérations sur les valeurs

Algorithme pour le calcul de la politique optimale π^* pendant la phase hors-ligne de planification à partir du MDP.

Value iteration [Bellman, 1957]

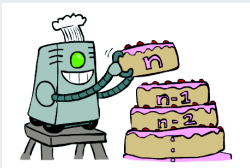
Évaluation itérative de V^* : calcule $V_0 \rightarrow V_1 \rightarrow V_2 \dots$

- ≡ Initialisation arbitraire de $V_0(s) \forall s$
- ≡ Mise à jour de $V_k(s) \forall s$ en utilisant les valeurs estimées à $k - 1$ des états voisins (*bootstrap*)

$$V_k(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_{k-1}(s')]$$

- ≡ Répète jusqu'à convergence

$$\text{critère d'arrêt } \max_{s \in S} |V_k(s) - V_{k+1}(s)| < \epsilon$$

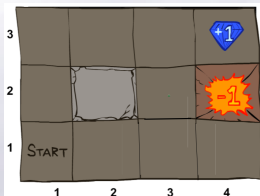


- ≡ théorème du point fixe de Banach : convergence de l'algorithme vers V^* solution de l'équation de Bellman pour tout V_0
- ≡ complexité de chaque itération $O(S^2 A)$

Itérations sur les valeurs : Illustration

$\forall s$ non absorbant

$$V_k(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_{k-1}(s')] \quad \gamma = 0.9$$



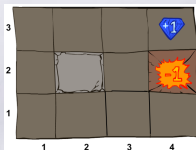
Exercice dans un labyrinthe déterministe

- ▢ Itération 0, $k = 0$, $V_k(s) = V_0(s) = 0 \forall s$
- ▢ Itération 1, $k = 1$, $V_1(s)$?
- ▢ Itération 2, $k = 2$, $V_2(s)$?
- ▢ Itération 3, $k = 3$, $V_3(s)$?

Itérations sur les valeurs : Illustration

$\forall s$ non absorbant $\gamma = 0.9$

$$V_k(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_{k-1}(s')]$$



0,0000	0,0000	0,0000	0,0000
0,0000		0,0000	0,0000
0,0000	0,0000	0,0000	0,0000

V_0

Exercice dans un labyrinthe stochastique

$$V_0(s) = 0 \quad \forall s$$

Calculer $V_1(s)$ et $V_2(s)$ (attention, il y a maintenant plusieurs états d'arrivée (s') pour un s et a !)

Itérations sur les valeurs

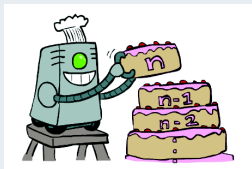
Value iteration [Bellman, 1957]

Évaluation itérative de V^* : calcule $V_0 \rightarrow V_1 \rightarrow V_2 \dots$

- ≡ Initialisation arbitraire de $V_0(s) \forall s$
- ≡ Mise à jour de $V_k(s) \forall s$ en utilisant les valeurs estimées à $k - 1$ des états voisins (*bootstrap*)

$$V_k(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_{k-1}(s')]$$

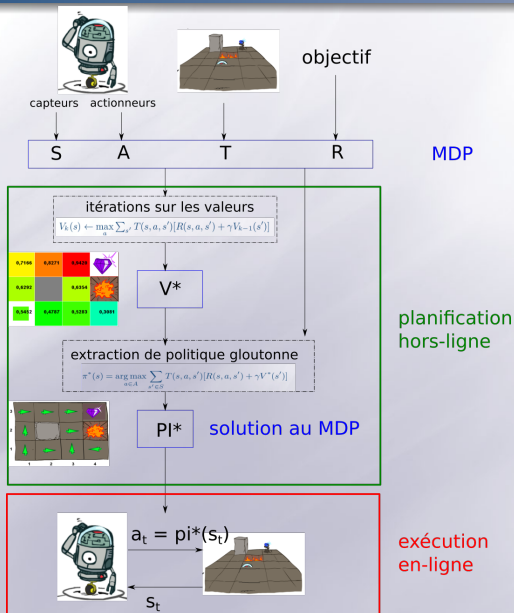
- ≡ Répète jusqu'à convergence
- critère d'arrêt $\max_{s \in S} |V_k(s) - V_{k+1}(s)| < \epsilon$



- ≡ extraction de π^* gloutonne sur V^*

$$\forall s \in S \quad \pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Récapitulatif



Plan

- 1 Notion d'agent
- 2 Introduction à la décision séquentielle
- 3 Formalisation mathématique
 - Problème : Modèle MDP
 - Solution : Politique
 - Objectif : Politique optimale
- 4 Fonction de valeur
- 5 Résolution d'un MDP
- 6 Application à l'exploration

Contexte : défi robotique CAROTTE

5 équipes



PACOM



YOJI



CARTOMATIC



ROBOTS_MALINS



COREBOTS

Objectif : système robotisé autonome pour la cartographie et l'exploration d'un environnement dynamique et inconnu

Problématiques :

- mobilité
- localisation et cartographie (SLAM)
- détection d'objets
- décision : où aller pour explorer efficacement ?

Contexte : défi robotique CAROTTE

5 équipes



PACOM



YOJI



CARTOMATIC



ROBOTS_MALINS

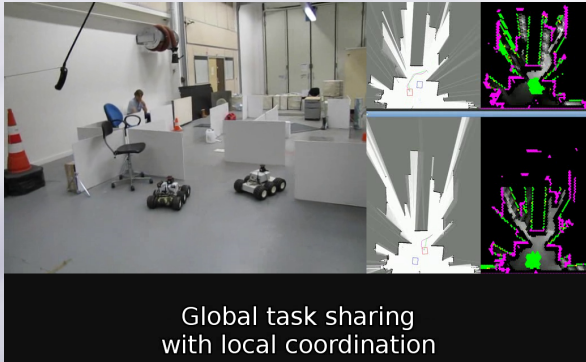


COREBOTS

- ≡ Objectif : système robotisé autonome pour la cartographie et l'exploration d'un environnement dynamique et inconnu
- ≡ Problématiques :
 - mobilité
 - localisation et cartographie (SLAM)
 - détection d'objets
 - **décision** : où aller pour explorer efficacement ?

Hypothèses

- Systèmes multi-robots : robots indépendants, pas de station centrale
- SLAM distribué/communication : chaque robot a accès à la carte fusionnée et aux localisations des robots
- reconnaissance “au fil de l’eau”



Développer des stratégies d'exploration multi-robot

Coordination décentralisée d'agents décisionnels :

- ≡ **coordination globale** : allocation des buts d'exploration (couverture efficace de la zone, minimiser les recouvrements).
 - ≡ **interactions locales** : à minimiser car exploration non efficace (recouvrements) et conflits nécessitant une coordination locale.
-
- ≡ Modèles de décision multi-agent : MDP multi-agent (MMDP, Dec-MDP, ...)
 - ≡ Résolution optimale très difficile (même pour 2 agents)
 - ≡ Proposition d'une méthode de résolution approchée

Extensions des MDP

Systèmes Multi-Agent (SMA)

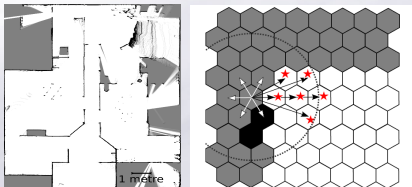
Dec-(PO)MDP [bernstein2002] $\langle n, S, A, T, R, \Omega, O \rangle :$

- n nombre de robots,
- $s \in S$ état **joint** du SMA $s = (s_1, s_2, \dots, s_n)$
- $a \in A$ action **jointe**
- $T : S \times A \times S \rightarrow [0; 1]$ fonction de transition des n robots d'un état **joint** s à s' après exécution de l'action **jointe** a
- $R : S \rightarrow \mathbb{R}$ fonction de récompense sur l'état **joint** s

MDP augmenté

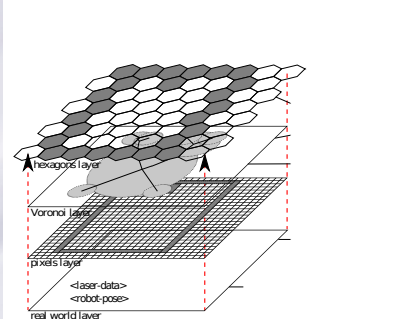
Notre approche

1. ensemble de MDPs locaux $\{MDP_1, ..., MDP_n\}$, un par agent.



MDP_i :

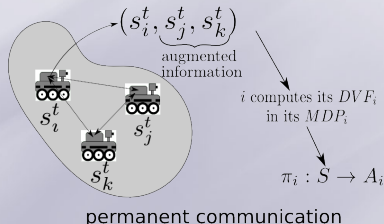
- S : 6 orientations $\times$ nombre d'hexagones
- A : avance, recule, tourne droite, tourne gauche + suivre voronoi
- T : état espéré suite à une action atteint avec une forte probabilité
- R : propagation des récompenses jusqu'à 2 mètres des frontières



Modèle MDP augmenté

Notre approche

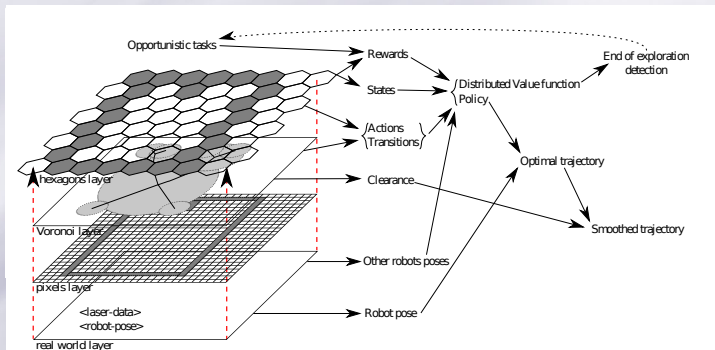
1. ensemble de MDPs locaux $\{MDP_1, \dots, MDP_n\}$, un par agent.
2. chaque MDP local est un **MDP augmenté** par des informations
3. l'information augmentée permet de prédire les intentions des autres par empathie



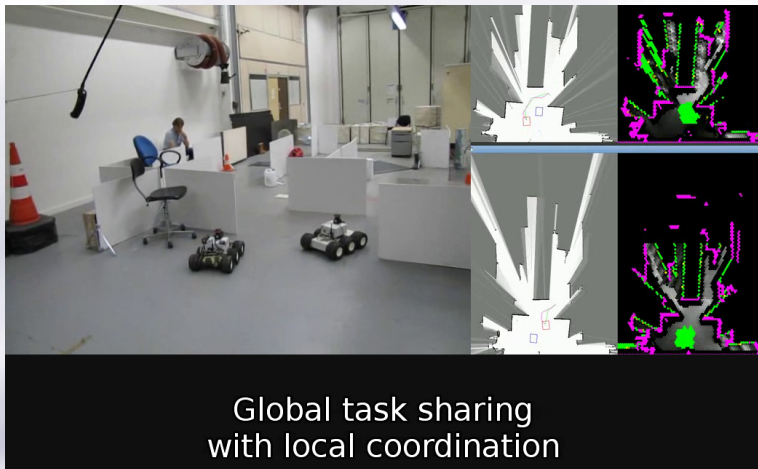
Modèle MDP augmenté

Notre approche

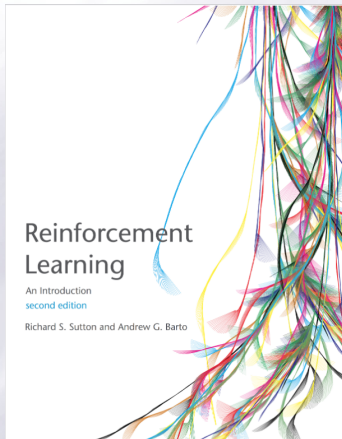
1. chaque robot résout par planification (*value iteration*) son MDP augmenté en utilisant l'information augmentée afin de minimiser les interactions
2. calcul d'un nouveau modèle et nouvelle politique toutes les secondes



Vidéos



Bibliographie



- Livre de Sutton & Barto: la bible du domaine
- David Silver Courses
- AI: A modern Approach, Stuart Russel and Peter Norvig